

Министерство науки и высшего образования Российской Федерации
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Алтайский государственный технический университет им. И.И. Ползунова»

Университетский технологический колледж

наименование подразделения

Кафедра Информационные системы в экономике

наименование кафедры

Направление Информационные системы и программирование

Отчёт защищен с оценкой _____

_____ К.В Воробьев

(подпись руководителя от вуза) (инициалы, фамилия)

“ _____ ” _____ 2025г.

ОТЧЕТ

по лабораторной работе №3

Статические методы и АТД

тема лабораторной работы

по дисциплине Объектно-ориентированное программирование

ЛР 09.02.07.28.001 ПЗ

обозначение документа

Студент группы _____ 1ИСП-22

_____ М.С. Заковряшин

инициалы, фамилия

Руководитель работы _____ преподаватель

должность, ученое звание

_____ К.В. Воробьев

инициалы, фамилия

Барнаул 2025

Создание абстрактного класса

- Создайте абстрактный класс.

```
class Figure(ABC):
    def __init__(self, color):
        self.color = color
        self.color_validation(color)

    @abstractmethod
    def can_move(self, board, row, column, row1, column1):
        return True

    @abstractmethod
    def can_attack(self, board, row, column, row1, column1):
        return self.can_move(board, row, column, row1, column1)

    @abstractmethod
    def get_color(self):
        return self.color

    @abstractmethod
    def color_validation(self, color):
        self.color = color.lower()
        if 'bl' in self.color or 'ck' in self.color:
            self.color = 'black'
            return color
        if 'w' in self.color or 'wh' in self.color:
            self.color = 'white'
            return self.color
```

Рисунок 1 - абстрактный класс фигура

- Наследуйте от абстрактного класса.

```

class Rook(Figure):
    def __init__(self, color):
        super().__init__(color)

    def can_move(self, board, row, column, row1, column1):
        return True

    def can_attack(self, board, row, column, row1, column1):
        return self.can_move(board, row, column, row1, column1)

    def get_color(self):
        return super().get_color()

    def color_validation(self, color):
        return super().color_validation(self.color)

```

Рисунок 2 - наследование от абстрактного класса

Статические поля и методы

- Придумайте, для чего в одном из ваших классов может быть разумным использование статического поля и статического метода.

```

class Chess_board:
    total_pieces = 32 # кол-во фигур
    created_boards = 0 #кол-во созданных досок

    def __init__(self):
        self.color = 'WHITE'
        self.field = []
        for row in range(8):
            self.field.append([None] * 8)
        self.field[0] = [
            Rook('WHITE'), Hourse('WHITE'), Bishop('WHITE'), Queen('WHITE'),
            King('WHITE'), Bishop('WHITE'), Hourse('WHITE'), Rook('WHITE')
        ]
        self.field[1] = [
            Pawn('WHITE'), Pawn('WHITE'), Pawn('WHITE'), Pawn('WHITE'),

```

Рисунок 3 - статические поля

- Создайте статические поля и методы.

```

@staticmethod
def display_board(board):
    for row in board:
        row_display = []
        for piece in row:
            if piece is None:
                row_display.append(' . ')
            else:
                row_display.append(f' {piece.__class__.__name__.lower()} ')
        print(' '.join(row_display))
    print()

```

Рисунок 3 - статический метод

- Продемонстрируйте их использование.

```

D:\УШАКОВ\chess_lr2\.venv\Scripts\python.exe "D:\Репозиторий\zakovryashin_1_isp_23_0\Лр 2\chess_zakovryashin_lr2_v0_2.py"
Цвет: black, тип фигуры Rook
Цвет: white, тип фигуры Rook
Цвет: black, тип фигуры Pawn
Цвет: white, тип фигуры Pawn
Наша доска:
rook  horse  bishop  queen  king  bishop  horse  rook
pawn  pawn  pawn  pawn  pawn  pawn  pawn  pawn
. . . . .
. . . . .
. . . . .
. . . . .
pawn  pawn  pawn  pawn  pawn  pawn  pawn  pawn
rook  horse  bishop  queen  king  bishop  horse  rook

Белые фигуры: 16
черные фигуры: 16
Создано досок: 2
фигуры 32 и 3 доски.
Исключение: division by zero

```

Рисунок 4 - демонстрация использования программы

Работа с динамическими структурами данных

- Замените массивы или списки на более подходящие структуры данных.

Массив и так подходит идеально

```

127     class Chess_board:
128         total_pieces = 32 # кол-во фигур
129         created_boards = 0 #кол-во созданных досок
130
131     def __init__(self):
132         self.color = 'WHITE'
133         self.field = []
134         for row in range(8):
135             self.field.append([None] * 8)
136         self.field[0] = [
137             Rook('WHITE'), Hourse('WHITE'), Bishop('WHITE'), Queen('WHITE'),
138             King('WHITE'), Bishop('WHITE'), Hourse('WHITE'), Rook('WHITE')
139         ]
140         self.field[1] = [
141             Pawn('WHITE'), Pawn('WHITE'), Pawn('WHITE'), Pawn('WHITE'),
142             Pawn('WHITE'), Pawn('WHITE'), Pawn('WHITE'), Pawn('WHITE')
143         ]
144         self.field[6] = [
145             Pawn('BLACK'), Pawn('BLACK'), Pawn('BLACK'), Pawn('BLACK'),
146             Pawn('BLACK'), Pawn('BLACK'), Pawn('BLACK'), Pawn('BLACK')

```

Рисунок 5 - работа с массивом

- Продемонстрируйте работу с этими структурами.

```

@staticmethod
def display_board(board):
    for row in board:
        row_display = []
        for piece in row:
            if piece is None:
                row_display.append(' . ')
            else:
                row_display.append(f' {piece.__class__.__name__.lower()} ')
        print(' '.join(row_display))
    print()

```

Рисунок 6 - работа с доской, ее вывод

Работа со строками

- Продемонстрируйте обработку строк.

```

test_figure = Rook('black')
print(f"Цвет: {test_figure.get_color()}, тип фигуры {test_figure.__class__.__name__}")

test_figure = Rook('white')
print(f"Цвет: {test_figure.get_color()}, тип фигуры {test_figure.__class__.__name__}")

test_figure = Pawn('blck')
print(f"Цвет: {test_figure.get_color()}, тип фигуры {test_figure.__class__.__name__}")

test_figure = Pawn('wh')
print(f"Цвет: {test_figure.get_color()}, тип фигуры {test_figure.__class__.__name__}")

```

Рисунок 7 - обработка строк

Перегрузка операторов

- Перегрузите оператор (минимум 3).

```

def __str__(self):
    return f" Фигуры {Chess_board.total_pieces} и {Chess_board.created_boards} доски."

def __eq__(self, other):
    if isinstance(other, Chess_board):
        return self.get_field() == other.get_field()
    return False

def __add__(self, other):
    if isinstance(other, Chess_board):
        new_board = Chess_board()
        new_board.field = self.get_field() + other.get_field() # сложение полей
        return new_board
    return NotImplemented

```

- Продемонстрируйте использование перегруженного оператора.

```

combined_board = board1 + board2 # доски сложили
print(combined_board) # информацию вывели жестко

```

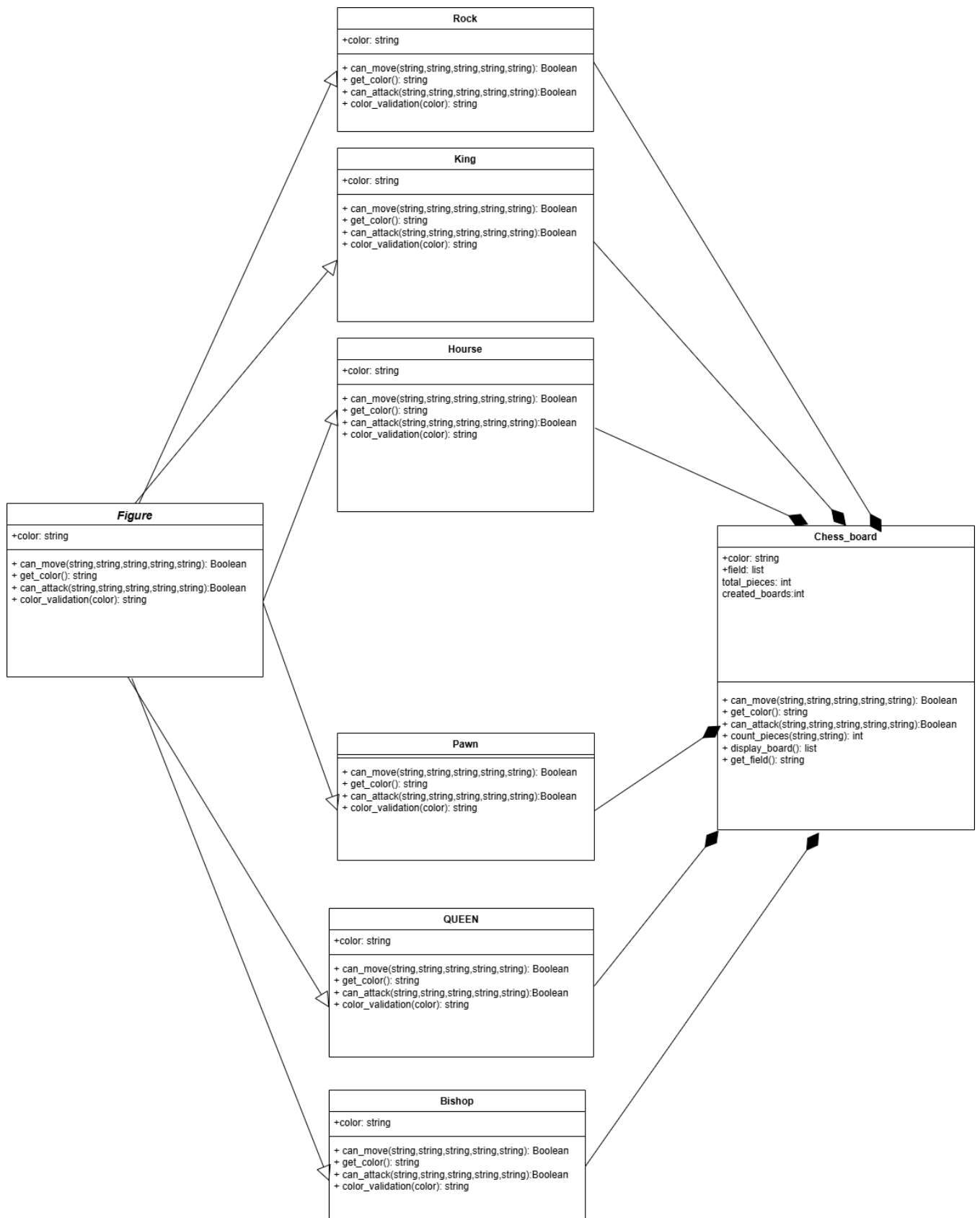
Рисунок 8 – использование перегруженных операторов

Дополнительные задачи

- Добавьте обработку исключений.

```
230     try:
231         1 / 0
232     except ZeroDivisionError as e:
233         print(f'Исключение: {str(e)}')
234
```

Рисунок 9 - обработка исключения



Вывод: в ходе лабораторной работы

https://github.com/LordFarqa/zakovryashin_1_isp_23_0/tree/object-oriented-programming